

Full Stack Engineer Interview Assignment

Water Network Editor with Role-Based Access and an Edit-Approval Workflow

Estimated effort ~3 days · React single-page app · No backend, browser-only state (Zustand or Redux)

1. Overview

You will build a single-page React application for reviewing and editing a water distribution network shown on a map. The application has three roles with different permissions, and every change to the network flows through a review-and-approval process before it becomes the published network that everyone sees.

There is no backend. All users, network data, edits, and history live in the browser using Zustand or Redux, persisted so that a refresh does not wipe a user's work. We care far more about how cleanly you model the roles, the approval workflow, and the change history than about visual polish.

2. The scenario

A water utility maintains a network of junctions, pipes, and valves. Network engineers propose changes such as a new pipe, a corrected diameter, or a valve setting. Before a change goes live, a field operator is often asked to verify something on site and record what they find, and a supervisor signs off on it. Your application models that loop from end to end.

3. Roles and permissions

Users are hardcoded in the frontend, at least one of each role. Logging in as a user determines what that user can see and do. No real authentication or password checking is required.

- **Admin** — reviews pending edits and approves or rejects them. An approved edit becomes the new published (default) network. Admins focus on governance rather than editing.
- **Editor** — creates and modifies network elements and their properties, and can dispatch a field task to an operator with instructions. Submits edits for approval.
- **Operator** — receives assigned field tasks and fills out a field form (measured or observed values plus comments) from the field. Cannot edit the network directly.

Capability	Admin	Editor	Operator
View the map and network	✓	✓	✓
Edit element properties	—	✓	—
Add or remove elements	—	✓	—
Assign a field task to an operator	—	✓	—

Capability	Admin	Editor	Operator
Fill and submit the field form	—	—	✓
Submit an edit for approval	—	✓	—
Approve or reject an edit	✓	—	—
Publish the approved edit as the default	✓	—	—
Post to the conversation thread	✓	✓	✓

4. The core workflow

1. An **Editor** logs in, selects an element on the map (or adds one), and changes its properties. These changes are grouped into a single **Edit** — a proposed change set — in a Draft state.
2. If field verification is needed, the Editor **assigns the Edit to an Operator** with a comment describing what to check.
3. The **Operator** logs in, sees the assigned task, and submits the **field form** — the measured or observed values plus any comments. This is captured against the Edit.
4. The Editor submits the Edit for review and it moves to **Pending Approval**.
5. An **Admin** reviews the Edit — the proposed property changes, the field input, and the comment thread — and **approves or rejects** it.
6. On approval, the Edit is **merged into the published network** and becomes the default that all roles see. On rejection, it returns to the Editor with the reason recorded.
7. Throughout, the Edit carries a **conversation thread and an audit trail** — who did what, when, and the field input that was recorded.

5. Functional requirements

5.1 Authentication and RBAC

- A login screen. Users are hardcoded (at least one Admin, one Editor, one Operator). Selecting or logging in as a user is enough; no password checks are needed.
- Actions are gated by role. An Operator cannot edit properties or approve, an Editor cannot approve, and only an Admin approves and publishes. Enforce this in the state and logic, not only by hiding buttons.

5.2 Network map

- Render the network on a map: at minimum junctions, pipes, and valves (reservoirs and tanks are optional). Elements are positioned by coordinates.
- Selecting an element shows its properties. The fields each element carries are shown in the sample data in section 7. You do not need to run any hydraulic simulation.

- Any mapping approach is fine — react-leaflet, MapLibre, Mapbox, or even an SVG or canvas rendering.

5.3 Editing the network

- An Editor can modify element properties — for example a pipe's diameter, roughness, or status; a valve's setting; a junction's elevation or demand.
- An Editor can add elements — a junction, valve, pipe, reservoir, or tank — and delete them. Removing an element updates or removes whatever depended on it; deleting a junction, for instance, handles the pipes attached to it.
- Adding a junction onto an existing pipe splits that pipe: the original pipe becomes two pipes that meet at the new junction, each still connected to one of the original endpoints. Both segments keep the original pipe's properties (diameter, roughness, status), and their lengths reflect the split point.
- For a sense of how a network editor handles adding, connecting, and editing elements on a map, refer to epanet-js (<https://epanetjs.com/>). It is a reference for the editing interactions, not a scope to match.
- Changes are collected into a single Edit rather than applied straight to the published network.

5.4 Field task and operator form

- An Editor can assign an Edit to an Operator along with a free-text instruction or comment.
- The Operator fills out a field form for that task — a small set of fields such as observed value, condition, and notes, plus a comment. The submission is recorded against the Edit and visible to the Editor and Admin.

5.5 Approval and the default layer

- Pending edits are queued for the Admin to review.
- On approval, the Edit's changes become the published or default network shown to everyone. Until then, the published network is unchanged.
- On rejection, the Admin's reason is recorded and the Edit goes back to the Editor.

5.6 Conversation and audit trail

- Each Edit has a threaded conversation that any role can post to, subject to their access.
- Each Edit has an audit trail: created, assigned, field form submitted, submitted for approval, approved or rejected — each entry with the actor and a timestamp. The recorded field input is part of this history.

6. Technical constraints

- React, with any tooling you prefer (Vite, CRA, or Next in SPA mode).
- No backend and no external persistence. State lives entirely in the browser via **Zustand or Redux**, persisted (for example with zustand persist or redux-persist) so that a page refresh preserves users, edits, and history.
- TypeScript is preferred but not required.

- Any UI or component library is fine.

7. Suggested starting data

You can mock a small network as a GeoJSON FeatureCollection — point features for junctions, valves, and reservoirs, and line features for pipes. This is the same format as the open data in section 12, so you can build against this and swap in real data later. Coordinates are illustrative and given as [lng, lat]:

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature", "properties": { "id": "R1", "type": "reservoir", "head": 250 },
      "geometry": { "type": "Point", "coordinates": [77.5930, 12.9700] } },
    {
      "type": "Feature", "properties": { "id": "J1", "type": "junction", "elevation": 210, "demand": 5 },
      "geometry": { "type": "Point", "coordinates": [77.5946, 12.9716] } },
    {
      "type": "Feature", "properties": { "id": "J2", "type": "junction", "elevation": 208, "demand": 8 },
      "geometry": { "type": "Point", "coordinates": [77.5966, 12.9726] } },
    {
      "type": "Feature", "properties": { "id": "J3", "type": "junction", "elevation": 205, "demand": 3 },
      "geometry": { "type": "Point", "coordinates": [77.5986, 12.9736] } },
    {
      "type": "Feature", "properties": { "id": "V1", "type": "valve", "valveType": "PRV", "diameter": 250, "setting": 40, "status": "active" },
      "geometry": { "type": "Point", "coordinates": [77.5976, 12.9731] } },
    {
      "type": "Feature", "properties": { "id": "P1", "type": "pipe", "start": "R1", "end": "J1", "length": 500, "diameter": 300, "roughness": 130, "status": "open" },
      "geometry": { "type": "LineString", "coordinates": [[77.5930, 12.9700], [77.5946, 12.9716]] } },
    {
      "type": "Feature", "properties": { "id": "P2", "type": "pipe", "start": "J1", "end": "J2", "length": 350, "diameter": 250, "roughness": 130, "status": "open" },
      "geometry": { "type": "LineString", "coordinates": [[77.5946, 12.9716], [77.5966, 12.9726]] } },
    {
      "type": "Feature", "properties": { "id": "P3", "type": "pipe", "start": "J2", "end": "J3", "length": 300, "diameter": 250, "roughness": 130, "status": "open" },
      "geometry": { "type": "LineString", "coordinates": [[77.5966, 12.9726], [77.5986, 12.9736]] } }
  ]
}
```

8. Deliverables

- A Git repository (or a zip) with the source and a README that covers how to run it, the seeded users and their roles, and any assumptions or trade-offs you made.
- The app should start with a documented command, for example `npm install` followed by `npm run dev`.
- Optional but appreciated: a short screen recording walking through the full flow across the three roles.

9. How we will evaluate

The core workflow, RBAC correctness, and how you model state carry the most weight. Polish is secondary.

What we evaluate	What strong work looks like
Workflow correctness	Edit states transition correctly, approved edits publish to the default network, and rejected edits return to the editor with the reason recorded.
Role-based access	Each role can do exactly what it should and nothing more, enforced in state and logic rather than only hidden in the UI.
State modelling & persistence	A clear store shape that keeps the published network, pending edits, and history separate. State survives a page refresh.
Map & interaction	Elements render in the right place and are selectable. Properties can be edited, and elements can be added, removed, and inserted onto a pipe (which splits it) while connectivity stays consistent.
History & audit trail	A complete, readable record of who did what and when, including the operator's field input.
Code quality	Sensible structure, naming, and componentisation. Typed where it helps.
UX clarity	A reviewer can move through the whole flow across all three roles without needing a walkthrough.

In the follow-up interview we will walk through your code together and ask you to extend it with a small new feature. You do not need to guess what it will be — the point is to see how you reason about and build on your own codebase, so a clean store and component structure will pay off.

10. Scope notes (already decided for you)

- No hydraulic simulation is required; the property fields are the ones shown in the sample data.
- No real authentication; hardcoded users are expected.
- Do not over-invest in styling.
- If something is ambiguous, make a reasonable assumption and note it in the README.

11. Stretch goals (optional, only if time allows)

- A diff view of a pending Edit showing proposed values against current ones.
- A map overlay that shows pending edits in a distinct style, with a toggle between the published and proposed layers.
- Undo and redo on edits.
- Search or filter for elements.
- A mock photo attachment on the field form.
- Import and export of the network as JSON.

12. Open data source

If you would rather build against real data than mock your own, the City of Vancouver open data portal publishes its water network as GeoJSON — the same FeatureCollection shape as the sample in section 7, with line features for mains and point features for valves and hydrants (the exact property names differ). It is city-scale, so filter to a small area (a bounding box) or render on a canvas layer rather than loading everything at once.

- **City of Vancouver open data** — <https://opendata.vancouver.ca/>. On each dataset page use Export → GeoJSON. Relevant datasets: water-distribution-mains, water-control-valves, water-hydrants.